National University of Computer and Emerging Sciences, Chiniot-Faisalabad Campus · Department of Software Engineering

Recommendation & Information Diffusion in a Social Network

SNTP — Assignment 2 · **Muhammad Mahad Bashir** · Roll No. **23F-3011** · 20 October 2026

Summary. I built one pipeline on the Epinions trust and rating data. It learns preferences with two recommenders written from scratch: user-kNN, which is memory-based, and biased matrix factorisation trained with ALS, which is model-based. One of them is extended with trust (SocialMF). Recommended items spread through the trust graph under an Independent Cascade model and an SIR epidemic model. CELF lazy greedy chooses whom to promote to under a budget and accounts for uncertainty. Groups are served by average, least-misery and most-pleasure aggregation. The system is evaluated on observed held-out ratings and, separately, in simulation. The main findings are:

- Trusted friends carry more useful preference information than similar strangers. SocialMF lowers test RMSE from 1.0630 to 1.0517 (3 splits). It helps most when the training history is shorter.
- Diffusion has a sharp critical point in the influence strength. Below it, the choice of seeds and the user preferences decide the outcome. Above it, everything spreads.
- Marginal-gain seeding beats degree seeding by up to 30%, and it should use preferences.
- Pushing broadly liked items maximises adoption but collapses diversity. Gating social exposure by predicted relevance trades a little spread for much less unwanted exposure.

1. System overview and evaluation philosophy

The platform is modelled in four layers: (i) **observed data**: who trusts whom and who rated what; (ii) **preference learning**: recommenders trained on the training split only; (iii) a **simulator**: diffusion over the trust graph, driven by a *simulated true preference* of every user for every item; (iv) **decision policies** that sit on top: seed selection, group recommendation, interventions. I keep two kinds of evidence strictly separate. **Observed outcomes** are held-out Epinions ratings. They support claims about real users, for example that SocialMF predicts real ratings better. **Simulated outcomes** are adoption counts, cascade speed and adopter satisfaction. They support only comparative, what-if conclusions under the stated model assumptions, for example that seed set A spreads more than seed set B *if* users behave like the model. Every simulated quantity is reported as a mean over repeated runs with its standard deviation or 95% confidence interval.

All code consists of plain Python functions (no classes). numpy/scipy are used only for array arithmetic; every recommender, simulator and selection algorithm is written by hand. Settings and seeds are in `config/config.json` (master seed 42, split seeds 0/1/2). The full run takes 6.2 minutes on a 12-thread laptop with 16 GB RAM (§9).

2. The social environment (Requirement 1)

2.1 Dataset and justification

I used the dataset supplied with the assignment: Epinions (Massa & Avesani 2007). It contains 664,824 ratings (1–5 stars) by 40,163 users on 139,738 items, and 487,183 directed trust statements. Epinions fits the task well because it is one of the few public datasets that holds *both* an explicit social network and user–item interactions for the same users. Users chose whom to trust after reading their reviews, which is exactly the "influence through friends" mechanism the scenario describes.

2.2 Meaning and direction of connections

A trust statement `A → B` means "A trusts B's reviews": B's opinions reach A. Two directions are therefore used. **For recommendation**, A's "friends" are the users A trusts (out-neighbours). Their tastes are evidence about A's taste. **For diffusion**, the edge is reversed to `B → A`. When B adopts and shares an item, A may see it, so B "influences" A, and B's out-degree in the influence graph is the number of people who trust B. Trust is asymmetric; only 47% of edges in the sample are reciprocated, so direction matters.

2.3 From interactions to preference signals

A rating is an **explicit, graded** preference. It is used as a numeric target for rating prediction (RMSE/MAE). It is also binarised as *relevant* when it is ≥ 4 stars, for ranking (precision/recall/NDCG). A **missing** rating is *unknown*, not negative. Users rate what they choose to consume, so the data are missing-not-at-random. For that reason I never treat unrated items as dislikes, and accuracy is defined only on observed ratings (§8).

2.4 Sampling and preparation (assumptions)

Monte-Carlo diffusion and greedy seeding on 40k users in Python would make the full set of repeated experiments impractically slow. I therefore work on a reproducible sample: (1) self-trust statements are removed (181); (2) items with ≥ 20 ratings are kept, together with users with ≥ 3 ratings on them who belong to the trust network. This leaves a core of 20,713 users, 5,540 items and 350,968 trust edges. (3) A **snowball (BFS) sample** of 3,000 users is drawn from a random start (seed 42). A uniform node sample would destroy almost all edges, because the expected number of edges kept falls with the square of the sampling rate. (4) Inside the sample, items with ≥ 5 ratings are kept. The result has **3,000 users, 4,736 items, 86,181 ratings (density 0.61%) and 136,331 trust edges**. The median user has 17 ratings, and 333 users have fewer than 5, so incomplete histories are naturally present. The trade-off is that a snowball sample is denser and more hub-heavy than Epinions as a whole, so absolute cascade sizes are not representative of the whole site. A 600-user demo profile uses the same code.

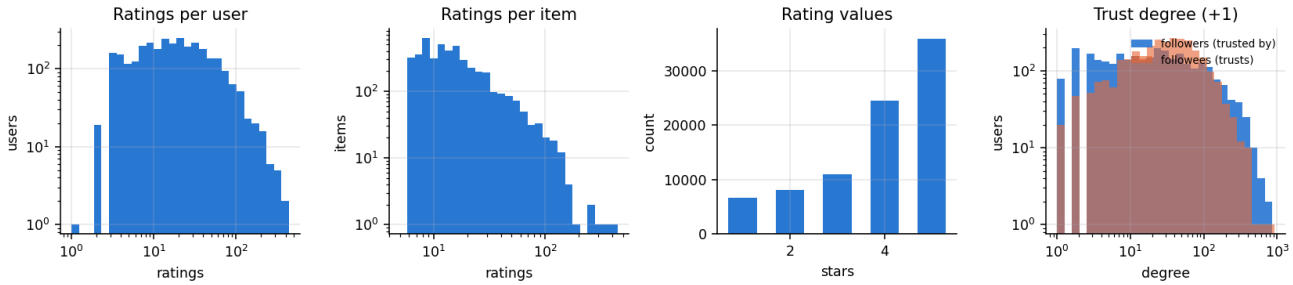


Figure 1. Sample statistics. Ratings per user and per item are heavy-tailed, ratings are skewed positive (70% are ≥ 4), and the trust degrees are heavy-tailed.

2.5 Observed vs. created/inferred information

Quantity	Status	How obtained
Ratings, trust edges, their direction	Observed	Epinions files
Relevance (rating ≥ 4)	Derived rule	Threshold on observed ratings
Edge strength s_{uv} and weight w_{uv}	Inferred	Rating agreement of the two users on co-rated *training* items: s = (#agree within ±1 star + 1)/(#co-rated + 2). The weights are normalised so that a user's total incoming influence is 1 (weighted cascade).
Global influence strength λ	Created (scenario)	Swept from 0.25 to 4
"True" preference of every user for every item	Inferred	The observed rating if it exists; otherwise the average of kNN and MF fitted on *all* observed ratings. Used only by the simulator.
Adoption probability $a_v(i)$	Created (model)	sigmoid(κ(pref − θ)), with κ = 2 and θ = 3.5
Cascades, exposures, adopter satisfaction	Simulated	IC / SIR runs

The varying strength of social influence enters in three ways: per edge (agreement s, see Fig. 1b in `results/`), per user (weighted-cascade normalisation: a user who trusts 200 people is influenced less by each of them), and globally through λ. **Observed evidence of social correlation:** a user is **5.2 times** more likely to rate (in the test split) an item that someone they trust rated in training (0.31% vs 0.06%). Trusting pairs share three or more co-rated items six times as often as random pairs (11.3% vs 1.9%). Their rating agreement is only slightly higher, however (0.747 vs 0.723). Trust predicts *what* people consume much more strongly than *how much they like it*, which foreshadows the modest RMSE gains in §4. Observational data like these cannot separate influence from homophily. That is one reason the diffusion results are framed as simulation.

3. Learning preferences and recommending (Requirement 2)

3.1 Approach A: user-based kNN (memory-based)

Each rating is converted to a residual $r_{ui} - b_{ui}$, where $b_{ui} = \mu + b_u + b_i$ is a regularised bias baseline. The residual removes a user's rating scale (harsh or lenient) and an item's general quality. Similarity is the cosine of the residual vectors, multiplied by the significance shrinkage $n_{common}/(n_{common}+10)$ so that similarities built on two co-ratings are distrusted. Only the k = 100 most similar users with positive similarity are kept. The predicted rating is $b_{ui} + \Sigma_v s_{uv}(r_{vi} - b_{vi}) / (\Sigma_v|s_{uv}| + d)$, where the damping d shrinks towards the baseline when little neighbour evidence exists. For **ranking** I use the classic top-N kNN rule: score = sum of similarities of neighbours who rated the item ≥ 4. A predicted rating is a poor ranking signal, because one weak neighbour's 5-star rating yields the same prediction as twenty strong neighbours' 5-star ratings. Similarities are computed in chunks of 1,000 users, so memory grows linearly rather than quadratically.

3.2 Approach B: biased matrix factorisation with ALS (model-based)

The model is $\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$. I train it with **Alternating Least Squares**: with item vectors fixed, every user's $[p_u, b_u]$ is the closed-form solution of a ridge regression, and vice versa. I wrote the solver to build all users' normal equations at once with one sparse–dense product and to solve them as a batch, so an iteration costs milliseconds. ALS was preferred over SGD because it has no learning rate to tune, its per-step solutions are exact, and it extends naturally to the social term in §4. Factors are regularised with λ_f and biases with λ_b = 3. With 0 factors the same code gives the bias baseline.

The two approaches are substantially different. kNN predicts from *explicit neighbours* and stores no learned model beyond the similarity graph. MF compresses all interactions into *latent dimensions*. kNN can explain

each recommendation; MF generalises through shared factors.

3.3 Hyper-parameters (no test leakage)

All tuning used a validation split (10% of each user's *training* ratings, split seed 0). Grids: kNN $k \in \{20, 50, 100\}$ and $d \in \{1, 5\}$; MF factors $\in \{5, 10, 15\}$ and $\lambda_f \in \{5, 10, 20\}$; SocialMF $\lambda_f \in \{5, 10, 20\}$ and $\beta \in \{0, 1, 3, 10, 30, 100\}$. Chosen values: kNN $k = 100$, $d = 1$; MF 5 factors, $\lambda_f = 20$; SocialMF $\lambda_f = 10$, $\beta = 10$. Validation consistently preferred **strong** factor regularisation for plain MF: with $\lambda_f = 5$, validation RMSE is 1.12; with 20 it is 1.068. At that point the factors are almost zero and MF behaves like its bias terms. On data this sparse, latent factors fitted to individual users over-fit. §4 shows that trust can replace zero as the prior and make the factors useful again.

3.4 Results on observed held-out ratings (mean ± std over 3 random splits)

Model	RMSE	MAE	Bal. acc.	P@10	R@10	NDCG@10	Coverage	ILD	Novelty
Popularity	–	–	–	0.0126	0.0281	0.0256	0.004	0.892	4.37
Bias baseline	1.0567 ± .0021	0.819	0.706	0.0018	0.0041	0.0028	0.147	0.950	7.44
User-kNN	1.0546 ± .0022	0.817	0.707	**0.0228**	**0.0509**	**0.0431**	**0.520**	0.893	5.56
MF (ALS)	1.0566 ± .0022	0.819	0.706	0.0017	0.0040	0.0027	0.153	0.950	7.44
SocialMF	**1.0517 ± .0025**	0.820	**0.707**	0.0020	0.0044	0.0032	0.108	0.951	7.31
kNN, user-mean centring	1.2081	0.957	0.597	0.0244	0.0525	0.0449	0.477	0.886	5.58
kNN, no centring	1.2288	0.923	0.664	0.0310	0.0717	0.0594	0.322	0.872	5.38

Standard deviations of the ranking measures are ≤ 0.001 to 0.006 (`rec_summary.csv`). Accuracy is 0.761–0.765 against a majority-class rate of 0.699. ILD is intra-list diversity (1 − item cosine); novelty is mean self-information in bits.

Interpretation. (1) *Rating prediction*: every personalised model beats the global-mean predictor (RMSE about 1.27) by a wide margin, but most of that gain comes from the biases. Epinions ratings are skewed and sparse, and the user/item effects explain most of the predictable variance. (2) *Ranking*: the picture reverses. kNN's neighbour votes beat popularity on precision (+81%) and coverage (52% of the catalogue vs 0.4%). Models that rank by *predicted rating* (bias, MF, SocialMF) are poor top-N rankers on the full catalogue. They push rarely-seen items with high item bias, which few users ever rate, so those items cannot be hits. This is the known gap between minimising RMSE and producing good top-N lists under missing-not-at-random data: low error does not imply good ranking. (3) The kNN variants show a trade-off of their own. Without centring, ranking improves (P@10 0.031), because raw votes favour popular items that users actually rate. RMSE collapses (1.23), however, because harsh and lenient raters are mixed. **Removing rating-scale differences is essential for *prediction*; it is not what drives *ranking*.**

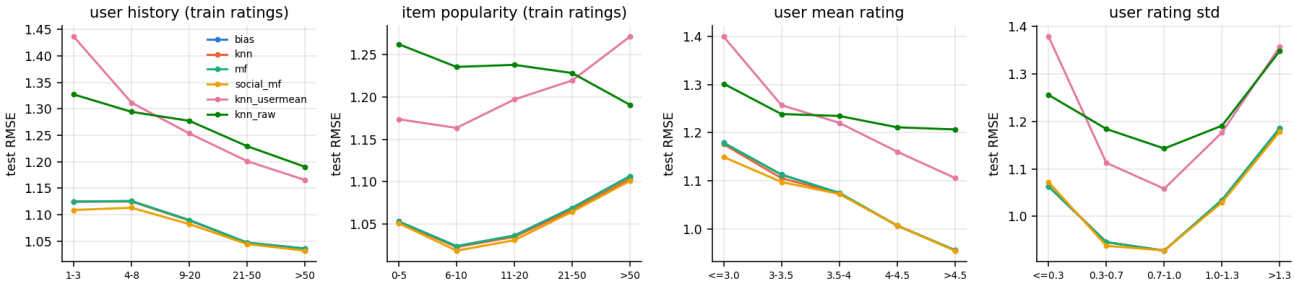


Figure 2. Test RMSE (pooled over 3 splits) by user history, item popularity, user mean rating and user rating spread.

3.5 Behaviour under limited histories, rare items and different rating patterns

- **Limited histories.** Error falls from 1.12–1.13 for users with 1–8 training ratings to 1.03 for users with more than 50. The cold bucket is where SocialMF gains most (1.109 vs 1.125). Users with few ratings have unreliable personal parameters, and a prior helps. Ranking quality also rises steeply with history: kNN NDCG is 0.023 for users with 1–3 ratings and 0.097 for those with more than 50, because similarity needs co-ratings.
- **Simulated cold start** (Fig. 3). I took 300 users with ≥ 20 training ratings, truncated their training history to $m \in \{1, 2, 3, 5, 10\}$ ratings, and evaluated them on their unchanged test ratings. kNN and MF equal the bias baseline at every m (1.0785 at m = 1): with $m \leq 10$ a user has almost no shrinkage-surviving neighbours and no estimable factors. **SocialMF is better at every m** (1.0609 at m = 1, −0.018), because it borrows the user's bias and taste from the people they trust.
- **Rarely rated items.** The 0–5 and 6–10 buckets are *not* the hardest. Their item biases are shrunk to the mean, which is a safe prediction. The most-rated items show the highest error (1.10), because opinions about popular items are more polarised.
- **Different rating patterns.** Harsh raters (mean ≤ 3) are hardest (1.15–1.18). Very lenient raters are easiest (0.96). Users with almost constant ratings (std ≤ 0.3) defeat neighbour methods: their centred vectors are near zero, so they have no similarity signal. User-mean centring without item effects fails badly for them (1.38). Erratic users (std > 1.3) are hard for every model.

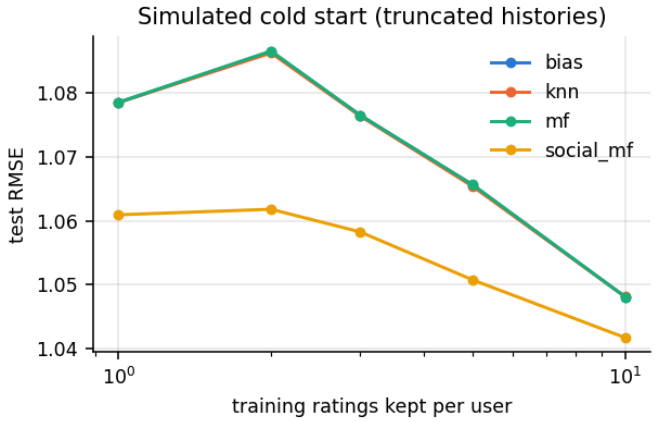


Figure 3. Simulated cold start. Only SocialMF departs from the bias baseline when a user has very few ratings.

4. Incorporating social relationships (Requirement 3)

4.1 SocialMF: a trust-regularised MF

I extended MF with the regulariser $\beta \cdot \Sigma_u \, \|x_u - \text{mean}_{f \in T(u)} \, x_f\|^2$, where $x_u = [p_u, b_u]$ and T(u) are the users u trusts (Jamali & Ester 2010; Ma et al. 2011). In the ALS user step this term simply adds βI to the normal matrix and β·(friends' average) to the right-hand side. The prior for a user therefore changes from "zero" to "what my trusted users look like". Users without friends are unaffected. **β is the knob that isolates the social contribution**: β = 0 is plain MF, and as $\beta \to \infty$ every user is forced to be the average of their friends.

4.2 Effect of the social weight

β	0	1	3	**10**	30	100
RMSE, all users	1.0630	1.0601	1.0561	**1.0517**	1.0555	1.0648
RMSE, ≤ 5 training ratings	1.1149	1.1082	1.1029	**1.1012**	1.1044	1.1077
RMSE, > 5 training ratings	1.0613	1.0584	1.0545	**1.0500**	1.0539	1.0634

Mean over 3 splits; the std is 0.002–0.004 for all users. $\lambda_f = 10$ throughout, so only β changes.

The curve is U-shaped (Fig. 4, left). Moderate trust regularisation improves every user group. It reduces error by 0.011 at the same λ_f, and it also beats the best-tuned plain MF (1.0566). The improvement is about 4× the split-to-split std, so it is small but systematic. At $\beta = 100$ the social term dominates the user's own ratings, and the model becomes *worse than no social information*. Friends are informative, but they are not the user.

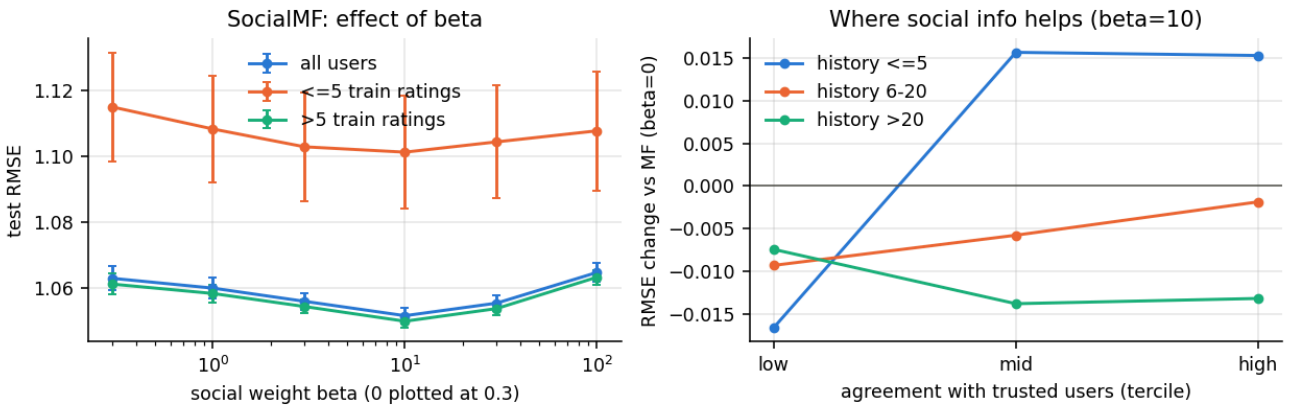


Figure 4. Left: test RMSE vs β (error bars = std over 3 splits). Right: change in RMSE relative to β = 0, by the user's agreement with trusted users (training data) and by history length. Negative means social information helps.

4.3 Do friends carry more information than similar strangers?

For every user u who trusts someone, I built three pools of the **same size** m = min(|T(u)|, 20) (mean 16): **friends** (users u trusts), **similar-unconnected** (the m users most similar to u in training who have no trust link with u in either direction), and **random-unconnected**. Each pool predicts u's *test* ratings in the same way (baseline + damped average of the pool members' residuals on the item), so only the source of information differs.

Pool	Residual corr. (train, ≥ 3 co-rated)	Pairs with ≥ 3 co-rated	Test pairs covered	Test RMSE where all pools cover
Friends	0.037	9.6%	40.8%	**1.1346**
Similar, unconnected	0.697	24.4%	38.6%	1.1486
Random, unconnected	−0.016	2.0%	14.6%	1.1881
(bias baseline, same pairs)				1.0701

The similar strangers *look* far more similar in training (correlation 0.70 vs 0.04). That number is circular, though, because they were *selected* for high training similarity. Out of sample, **friends predict better (1.135 vs 1.149)**, and both beat random users (1.188). Friends also cover more test items, because they consume the same things (§2.5). In-sample similarity over-fits to a handful of co-ratings, whereas trust is an independent and stable signal. The table also shows a caveat: **every unweighted pool is worse than the baseline**. Raw neighbour opinions are noisy and must be *shrunk*, which is exactly what SocialMF's regulariser and kNN's damping do.

4.4 When does social information help or hurt?

- **Helps** users with short histories (cold start, Fig. 3); users whose rating scale differs from the global one but resembles their friends' (the bias is shared through the prior); and moderate β. Gains are largest for users with low agreement and ≤ 5 ratings (−0.017), where the user's own data are weakest.
- **Hurts** when β is too large (β = 100 is worse than β = 0), and for cold users with medium or high agreement in my split (+0.015). For them the bias baseline was already accurate, and pulling them towards many friends added variance. This sub-group effect has the largest uncertainty (few test ratings), so I would not over-interpret it. Social information also cannot help the 0.7% of users who trust nobody in the sample.

- For **ranking**, SocialMF inherits MF's weakness (NDCG 0.003). Trust improves *how much* users will like items; it does not fix *which* items to show.

5. Adoption and information spread (Requirement 4)

5.1 Three distinct events

- **Receiving a recommendation.** The platform pushes item i to a seed user v at t = 0. v adopts with probability $a_v(i)$ = sigmoid(2($pref_{vi}$ − 3.5)), which is 0.95 at 5 stars, 0.73 at 4 and 0.27 at 3.
- **Exposure through a neighbour.** When u adopts and shares, each follower v notices the share with probability $attn_{uv}$ = min(1, λw_{uv}). Noticing makes v *exposed*. Exposure is counted whether or not v adopts.
- **Adoption.** An exposed user adopts with probability $a_v(i)$. Preferences therefore gate diffusion: social influence brings the item to v's attention, and v's own taste decides.

5.2 The two models: states and transitions

Independent Cascade (IC). The states are *inactive* (has not adopted) and *active* (has adopted). A newly active user gets **exactly one chance** to expose each follower in the next step, then stays active but silent. On a platform this is a single share appearing once in followers' feeds.
SIR epidemic. The states are **S** (has not adopted; can be exposed repeatedly), **I** (adopted and actively promoting: re-shares, mentions, talks about it each step) and **R** (adopted but has lost interest and no longer promotes). At each step every I user exposes their followers, then recovers with probability γ. The mean promoting period is therefore 1/γ steps; with γ = 0.5 it is 2. **With γ = 1, SIR is exactly IC**, and my implementation uses the same function for both. This makes the difference between the models precise: SIR allows *repeated exposure attempts over an infectious period*. For one edge the probability of transmission rises from p to $1 - (1-p)^T$. SIR therefore reaches the critical point at a lower λ, spreads further and spreads more slowly. (SIS, in which users can re-adopt, does not fit "adopting an item" and was not used.)

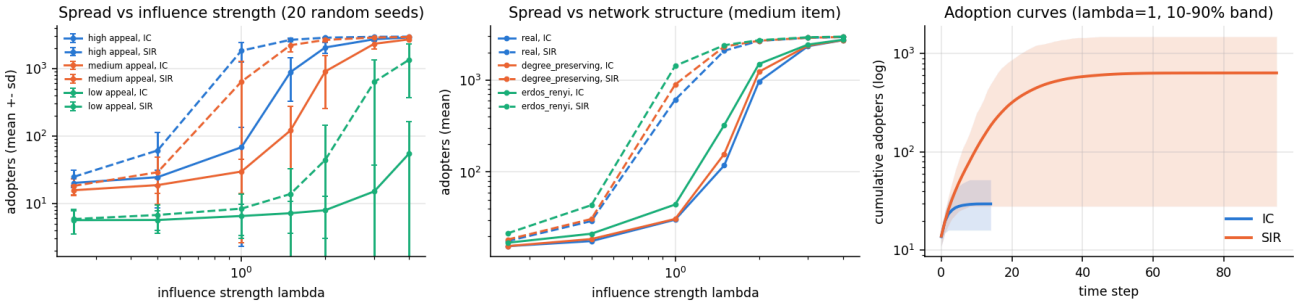


Figure 5. Left: adopters vs influence strength λ for items of high, medium and low mean appeal (mean adoption probability 0.87 / 0.70 / 0.26), from 20 random seeds and 200 runs per point; solid = IC, dashed = SIR (γ = 0.5). Middle: real vs degree-preserving rewired vs Erdős–Rényi graph. Right: adoption curves at λ = 1.

5.3 What drives size and speed (200 runs per setting)

- **Influence strength: a phase transition.** For the medium item under IC, mean adopters are 16, 19, 30, 120, 903 and 2,326 at λ = 0.25, 0.5, 1, 1.5, 2 and 3. Near the critical point (λ ≈ 1.5–2) the outcome is **extremely variable** (std 153 at λ = 1.5 and 646 at λ = 2): the same seeds either fizzle or go viral. A single simulation would be meaningless here, which is why every point is a 200-run average.
- **Model.** SIR with γ = 0.5 at λ = 1 reaches 637 adopters vs 30 for IC: the repeated exposures push it past the critical point. The γ sweep is monotone (γ = 1 → 32, 0.5 → 663, 0.25 → 2,606 at λ = 1). SIR is also slower: 90% of final adoption at step 22 vs 2.6 for IC. Its cascades last 33 steps vs 4.
- **Preferences.** Item appeal shifts the critical point. The low-appeal item stays local (8 adopters with IC at λ = 2, vs 903 for the medium item and 2,051 for the high-appeal item). Stronger preference gating (κ from 0 to 4 at fixed λ) raised spread from 17 to 43 adopters for this item, because enthusiasts adopt almost surely and form chains. Replacing personalised probabilities by their mean ("preference-blind") changed spread

only slightly at random seeds (30 vs 34). Preferences matter mostly through *where* the enthusiasts are relative to the seeds (§6).

- **Network structure.** The real trust graph spreads **less** than its randomised versions (IC at λ = 1.5: real 118, degree-preserving 157, ER 323). Real trust is clustered (clustering coefficient 0.26 vs 0.16 and 0.03) and reciprocal (47% vs 6% and 1.5%). Shares are therefore wasted on users who were already exposed, which is the "overlapping audiences" effect in the scenario. Above the critical point all graphs saturate the sample.

6. Choosing whom to promote under a budget (Requirement 5)

6.1 Strategy: CELF over live-edge worlds with uncertainty

Under IC, flipping every edge's coin in advance gives a **live-edge world**. Edge u→v is live with probability $attn_{uv} \cdot a_v(i)$, and a promoted user adopts on the recommendation with probability $a_v(i)$. The adopters are exactly the users reachable from adopting seeds. I sample R = 100 worlds **once** and evaluate every candidate on the same worlds. These common random numbers make candidate comparisons far less noisy than independent simulations. The **marginal gain** of candidate c, given the chosen set S, is the number of users c reaches that S does not already reach, averaged over worlds. A hub whose audience overlaps the current seeds' audiences scores low. Expected spread is monotone and submodular, so greedy selection achieves at least (1 − 1/e) of the optimum (Kempe et al. 2003). CELF lazy evaluation (Leskovec et al. 2007) re-evaluates only the top heap candidate, because its gain can only have shrunk.

Uncertainty. The per-world gains give each candidate's mean contribution and its standard error. I report the 95% CI at every greedy step (Fig. 6, right). I also implement an **LCB variant** that ranks candidates by mean − 1.96·SE, which prefers contributions that are *reliably* large. Final seed sets are always evaluated with **fresh, independent simulations** (300 runs), never on the worlds used for selection, because those worlds are optimistically biased.

Baselines under the same budget: *random* users (10 draws); *high degree*, the users trusted by the most people (a simple network baseline); and *top individual spread*, the k users with the largest stand-alone expected spread. The last one ignores overlap; it is exactly "relying only on individual popularity". The campaign item is the medium-appeal item.

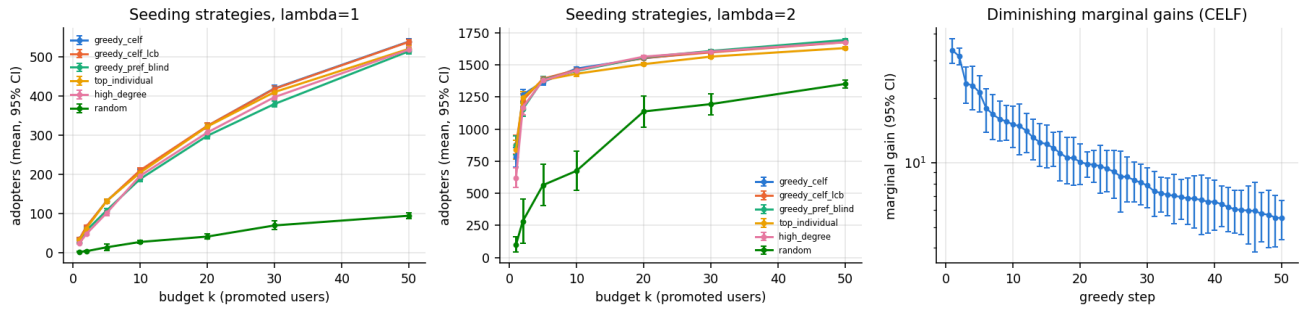


Figure 6. Adopters vs budget for each strategy (95% CI over 300 fresh runs) at λ = 1 (sub-critical) and λ = 2 (viral), and CELF's marginal gains with 95% CIs.

λ = 1, adopters (mean)	k=1	k=2	k=5	k=10	k=20	k=50	Audience overlap (k=5)
CELF (mean gain)	34.4	64.2	131.1	206.4	323.4	**539.8**	0.173
CELF (LCB)	34.8	66.0	130.6	**210.0**	**323.9**	539.1	0.173
CELF, preference-blind	25.8	55.9	107.0	187.9	298.6	514.9	0.139
Top individual spread	32.5	61.3	131.2	204.0	322.1	521.2	0.173
High degree	23.9	47.3	100.5	196.0	306.9	519.2	0.245
Random	1.5	3.1	13.1	26.8	40.6	93.9	0.009

The 95% CI half-widths are 2.7–7.6 adopters for the deterministic strategies (`im_budget_comparison.csv`).

- **Against the baselines.** CELF beats high degree by 44% at k = 1, 30% at k = 5 and 4% at k = 50, with non-overlapping CIs for k ≤ 5. It beats random by roughly 6–22 times. The most-trusted users are not the best starting points: their audiences overlap more (Jaccard 0.245 vs 0.173 at k = 5), and some of them dislike the item.
- **Overlapping audiences.** Redundancy grows with the budget, measured as 1 − joint spread / sum of individual spreads: 0.04 at k = 5, 0.18 at k = 20 and 0.31 at k = 50 for CELF. Top-individual seeding ignores this and falls behind as k grows (521 vs 540 at k = 50, with redundancy 0.37). The marginal gain per extra seed falls from 34 to 5.5 adopters (Fig. 6), which is diminishing returns.
- **Budget and regime.** In the viral regime (λ = 2) one good seed already triggers about 800 adopters. By k = 5 every sensible strategy saturates at about 1,380 (redundancy 0.65), and only random seeding still lags. Smart seeding matters most for **small budgets near or below the critical point**.
- **Preferences matter for seeding.** CELF run on preference-blind worlds loses 5–25% of spread: it seeds users whose neighbourhoods do not like the item.
- **Uncertainty.** With R = 10 worlds the selection's own estimate is optimistic by 29% (240 estimated vs 187 achieved, mean of 3 repetitions), and the chosen sets are worse (achieved 187 vs 208 with R = 100). With R = 100 the bias falls to about 4% and the first-step CI half-width drops from about 25 to about 4. At λ = 2 and k = 1 the LCB variant achieved more than mean-gain CELF (879 vs 783, within overlapping CIs), because it avoided a candidate whose large mean came from a few explosive worlds.

7. Recommending to groups (Requirement 6)

Design. Groups are a user plus three of their trust contacts (friends deciding together). I used a *nominate-then-decide* protocol: each member nominates their personal top-5 (kNN) among items no member has rated and that have ≥ 10 ratings, and the group picks one nomination by aggregating SocialMF predictions. A member who rates everything lower shifts all scores equally and never changes the chosen item. Conflict is therefore measured on *user-centred* predictions: the mean across-member std of relative preferences over the nominations. The 40 most conflicted of 400 groups were kept. In 68% of them the three strategies choose different items. Satisfaction is evaluated with the simulated true preferences. Spread is simulated by seeding all four members with the chosen item (IC, λ = 1, 50 runs per item) and comparing it with every other candidate in the union of the strategies' top-5 lists.

Strategy	Mean pref.	Least happy member	Happiest member	Members ≥ 4	Adopters via group	Is the max-spread item	Spread / best	Spearman(score, spread)
Average	**4.62**	**4.25**	4.94	**95.6%**	**66.3**	45%	**0.91**	**0.55**
Least misery	4.60	4.19	4.93	93.8%	64.2	**48%**	0.88	0.46
Most pleasure	4.43	4.02	4.82	85.0%	53.8	18%	0.72	0.33

Std over groups: mean preference 0.16–0.27; adopters about 37 (`group_summary.csv`).

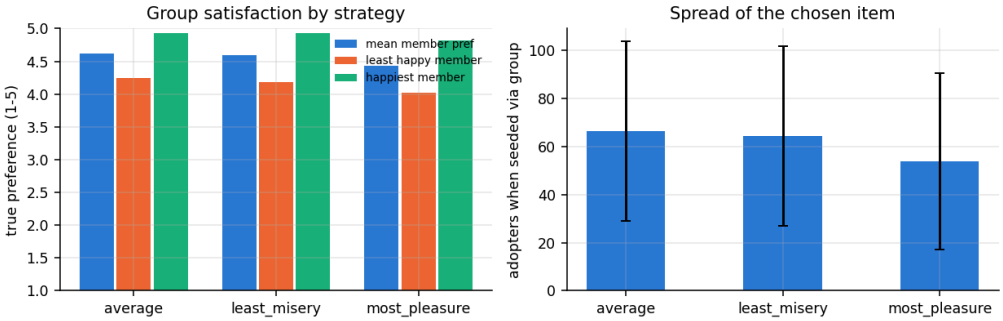


Figure 7. Group satisfaction (simulated true preference) and the spread of each strategy's item when introduced through the group.

Interpretation. Most pleasure follows one enthusiast's prediction. It is the most vulnerable to that member's prediction error and to the others' indifference, so it gives the lowest satisfaction for everyone, including the happiest member. Average and least misery are close. Least misery protects the worst-off member in theory, but in practice it is slightly *worse* on that measure (4.19 vs 4.25), because a minimum over noisy predictions picks items whose low score is under-estimated. **Is the preferred item also the most viral?** Only partly. The average strategy's item is the max-spread candidate in 45% of groups and achieves 91% of the best spread on average. The correlation between group score and spread is positive but moderate (0.55). It is positive because the group's own members are the first adopters, and spread beyond them depends on the *wider neighbourhood's* tastes, which the group score does not see. Most pleasure's item spreads least: an item loved by one member and lukewarm for the rest loses three of the four initial adopters.

8. Evaluating the complete system (Requirement 7)

8.1 Evaluation design

- **Data split.** A random 20% of *each* user's ratings is held out (at least one per user with ≥ 3 ratings), repeated for 3 split seeds; results are mean ± std. Every user in the test set has training history.
- **Leakage prevention.** Similarities, biases, factors, hyper-parameters (on a validation split carved out of training), edge strengths and the friend-agreement features are all computed from training ratings only. Items a user rated in training are excluded from their lists.
- **Relevance and cut-off.** A test rating ≥ 4 is relevant. K = 10, one screen of recommendations.
- **Candidate items.** All items the user did not rate in training (full ranking, no negative sampling). Absolute precision is therefore small (about 1 relevant test item among 4,700 candidates), but the comparison between models is unbiased by the choice of sampled negatives. For *promotions*, only items with ≥ 10 training ratings are eligible.
- **Accuracy.** The task is classifying observed test ratings as liked (≥ 4) or not liked (≤ 3) by thresholding the prediction at 3.5. **Negatives are observed low ratings; unrated items are never negatives.** Balanced accuracy is reported because 70% of ratings are positive (majority-class accuracy 0.699).
- **Measures.** Prediction error: RMSE, MAE. Ranking: precision, recall, NDCG, hit rate @10. Beyond accuracy: catalogue coverage, intra-list diversity, novelty, Gini. Diffusion: adopters, social exposures, seed adoption, time to 90% of the final size, with std and 95% CI over repeated runs.

8.2 Interaction: recommendations that start cascades

100 random target users (3 independent draws) each receive their top-1 eligible recommendation from a given method. Every distinct item then spreads independently from the targets who received it (IC, $\lambda = 1$, 30 runs). The "hybrid" scores items by $(1-\eta)\cdot$personal score $+ \eta\cdot$social appeal, where social appeal is the predicted adoption probability of the target's followers weighted by influence. It uses only the recommender, not the hidden truth.

Method	Distinct items	Targets' true pref.	Targets with pref ≥ 4	Observed hit (test)	Total adopters	Adopters' pref.	Unwanted exposures
Popularity	3.0	4.11	64%	2.7%	342 ± 24	4.30	4.8
User-kNN	**49.7**	4.23	73%	**3.0%**	456 ± 70	4.47	3.0
MF	5.3	4.89	99.7%	0.3%	732 ± 35	4.88	0
SocialMF	4.3	4.89	99.7%	0.7%	729 ± 79	4.89	0
Hybrid η = 0.5	2.3	4.89	99.7%	0.7%	592 ± 41	4.90	0

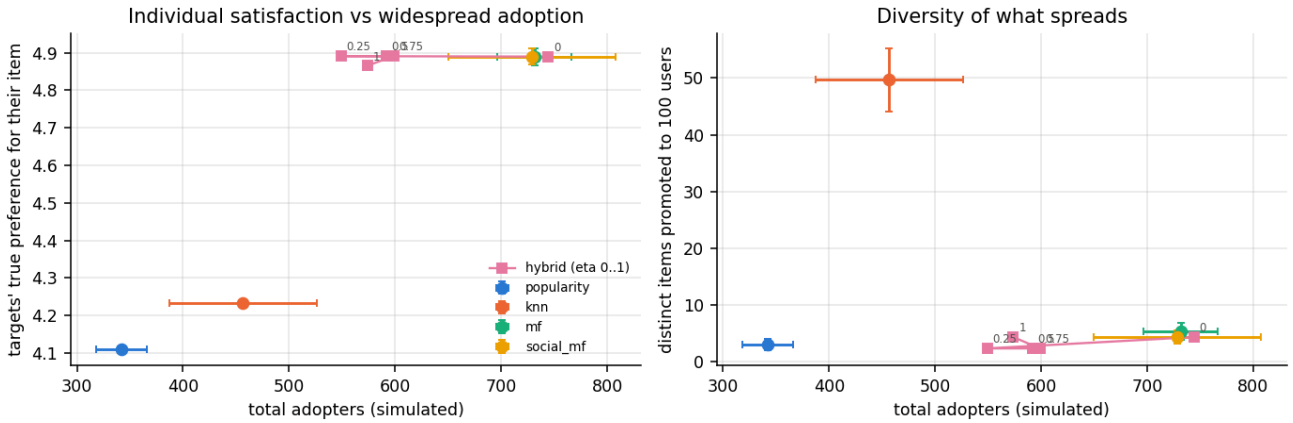


Figure 8. Left: targets' satisfaction vs total simulated adoption. Right: the number of distinct items that a campaign of 100 recommendations ends up spreading.

Simulated vs observed conclusions disagree, and the reason is instructive. In the *simulator*, MF and SocialMF look best on both satisfaction and spread. They recommend the same 4–5 near-universally liked items to almost everyone, so the item cascades merge. But the simulator's "truth" is itself built from kNN and MF predictions. It inherits their bias towards items with a high item bias, so these numbers **overstate** MF-style recommendations. The *observed* test data say the opposite: kNN's recommendations are actually rated ≥ 4 by the targets 10 times more often than MF's (3.0% vs 0.3%), and kNN covers 50 different items instead of 5. This is the clearest case in the report of why observed and simulated outcomes must be kept apart. Conclusions about real satisfaction rest on the observed hits; the simulator only supports statements such as "concentrating on consensus items maximises cascade size".

The **diversity–adoption trade-off** is robust under both lenses. Spreading many personalised items (kNN: 50 items) yields fewer total adoptions than pushing a few consensus items, because cascades of the same item reinforce each other. The spread-aware hybrid did not increase spread (592 at η = 0.5 vs 744 at η = 0). Its one-hop "appeal" signal is a poor proxy for multi-hop cascades. That is an honest negative result.

9. Intervention and computational cost (Requirement 8)

9.1 Interventions that limit diffusion

Scenario. The medium-appeal item (only 54% of eventual adopters truly like it, pref ≥ 4) is pushed to the 20 most-followed users in the viral regime (λ = 2, 100 runs). Three interventions are compared: a **fan-out cap** (a share reaches at most m followers); a **relevance gate** (a share is shown only to followers whose *predicted* preference, from SocialMF, is ≥ τ; the gate cannot see the true preferences); and **account blocking** (the b most-followed users, or b random users, cannot share).

Intervention	Adopters	Social exposures	Adopters truly satisfied (≥ 4)	Unwanted exposures (true pref < 3)
None	1,564 ± 115	1,965	53.6%	17.3
Fan-out cap 10 / 5 / 2	20 / 16 / 14	10 / 4 / 2	52% / 51% / 50%	0.04 / 0.03 / 0.02
Relevance gate τ = 3.0	1,559	1,959	53.7%	17.0
Relevance gate τ = 3.5	**1,391 (−11%)**	1,736	55.7%	**4.9 (−72%)**
Relevance gate τ = 4.0	221 (−86%)	250	**92.1%**	0.2
Block top-degree 10 / 50 / 100	1,464 / 1,189 / 963	1,859 / 1,541 / 1,269	54% / 54% / 55%	17.0 / 13.5 / 11.4
Block random 10 / 50 / 100	1,540 / 1,495 / 1,451	1,937 / 1,889 / 1,838	54% / 54% / 54%	17.8 / 17.1 / 16.3

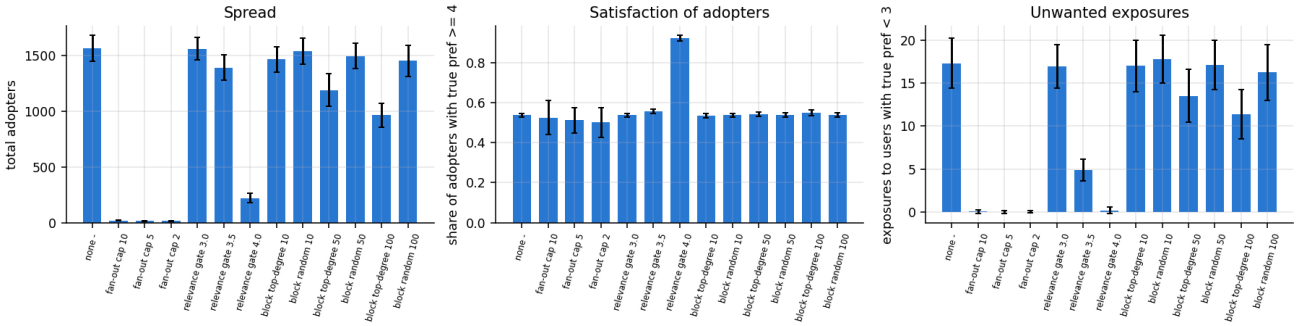


Figure 9. Spread, adopters' satisfaction and unwanted exposures under each intervention (error bars = std over 100 runs).

Effects on spread and satisfaction. A fan-out cap is a blunt instrument. In this hub-seeded scenario it pushes diffusion below the critical point and kills almost all spread. Unwanted exposures disappear, but so does all secondary adoption, and the share of satisfied adopters does not improve. Blocking hubs reduces spread far more than blocking random accounts (963 vs 1,451 at b = 100), which confirms the structural role of hubs, but it does not change *who* adopts, so satisfaction is unchanged. The **relevance gate** is the only intervention that improves satisfaction. At τ = 3.5 it removes 72% of unwanted exposures for an 11% loss of adoption. At τ = 4 it turns the campaign into a small cascade among enthusiasts: 92% of adopters are satisfied, but spread falls by 86%. This is the satisfaction–reach trade-off in its purest form, and τ is the dial that controls it.

9.2 Runtime and memory scaling

I grew the network from 500 to 8,000 users (fresh snowball samples), the catalogue from 250 to 4,000 items (top-I items of the main sample), and the budget from 1 to 40. Runtime was measured without tracing; peak memory was measured in a separate run with `tracemalloc`.

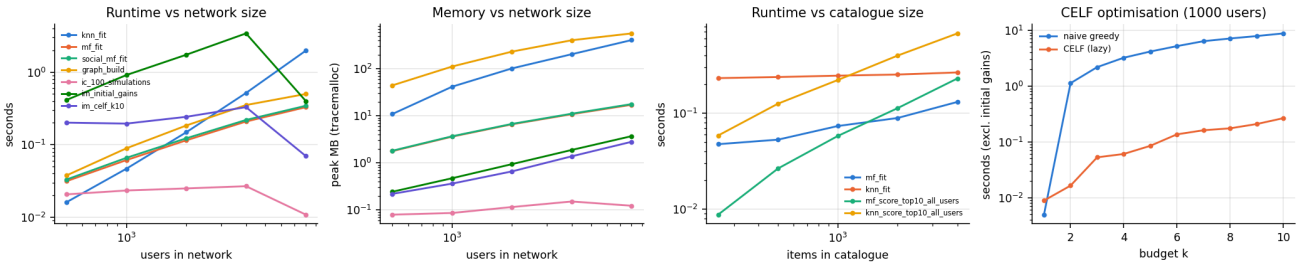


Figure 10. Runtime and peak memory vs network size; runtime vs catalogue size; naive greedy vs CELF (same 50 worlds, 1,000 users).

Operation	500 users	2,000	4,000	8,000	Growth
kNN similarity (time / peak MB)	0.02 s / 11	0.15 s / 102	0.52 s / 204	1.98 s / 407	time ≈ N^2; memory ≈ N (chunked)
MF / SocialMF fit	0.03 s / 1.8	0.12 s / 6.7	0.21 s / 11	0.33 s / 17	≈ linear in ratings
Influence graph (edge agreement)	0.04 s / 44	0.18 s / 232	0.35 s / 404	0.50 s / 564	linear in edges; **largest memory**
100 IC simulations	0.02 s	0.03 s	0.03 s	0.01 s	depends on cascade size, not N
IM initial gains (50 worlds)	0.41 s	1.73 s	3.41 s	0.40 s*	≈ N × R × cascade

*The 8,000-user sample's campaign item (its most-rated item) has a small cascade, so the diffusion steps get cheaper. Their cost scales with the size of the reachable sets, not with the number of users. Catalogue growth: scoring and ranking all users grows linearly in I (MF 0.009 s to 0.23 s, kNN 0.06 s to 0.68 s from 250 to 4,000 items); kNN fitting is almost independent of I. Budget: CELF time grows roughly linearly in k (0.02 s at k = 1 to 1.1 s at k = 40 after the initial pass), with only 3,000 + 341 gain evaluations for k = 40.

9.3 Bottleneck and optimisation

The dominant cost is **Monte-Carlo greedy seed selection**. Naive greedy recomputes every candidate's marginal gain in every round: O(k·N·R·cascade). On the main network with k = 50, that is an estimated **147 s**, against at most 2 s for any other stage. I implemented and measured two optimisations:

- **CELF lazy evaluation.** On 1,000 users, 50 worlds and k = 10, naive greedy needs 9,955 gain evaluations and 8.75 s; CELF needs 1,068 evaluations and 0.26 s. That is a **33× speed-up with an identical seed set at every k** (verified), which follows from submodularity. Pruning the reachability search at already-covered users makes each evaluation cheaper as well.
- **One search per strongly connected component.** All users in the same SCC of a live-edge world reach the same set, so the initial gains need only one search per component. The gains are identical (verified). The speed-up is 1.45× in the viral regime (λ = 2), where a giant component exists, and none at λ = 1, where components are tiny.

Other choices were made with cost in mind: batched ALS solves (all users in one call), chunked kNN similarity (memory linear in N), and vectorised frontier-based simulation (about 0.1–0.3 ms per IC run). The remaining memory hot spot is the edge-agreement computation, which indexes sparse rows per edge (564 MB at 8,000 users). Processing edges in chunks would bound it.

10. Final discussion

When does combining recommendation with social influence improve the platform's outcomes?

- **When the user's own data are thin.** Trust is a stable, independent signal. SocialMF improves real (observed) rating prediction at every history length and helps most for cold users (Fig. 3). Friends predict better than similar strangers (1.135 vs 1.149 RMSE).
- **When the seeding is preference-aware and overlap-aware, and the budget is small near the critical point.** Marginal-gain seeding with preferences beat the "most-trusted users" heuristic by up to 44% and preference-blind greedy by up to 25%.
- **When social exposure is filtered by predicted relevance.** The relevance gate at τ = 3.5 kept 89% of adoptions while removing 72% of the exposures that bothered users who dislike the item.

When does it create a trade-off between individual satisfaction and widespread adoption?

- **Above the critical influence strength.** Cascades reach users regardless of taste (only 54% of adopters satisfied in the unchecked viral campaign). Containing them to the satisfied core cuts reach by up to 86% (relevance gate τ = 4).
- **When maximising adoption favours consensus items.** Personalised, diverse recommendations (kNN: 50 items, the highest *observed* hit rate) produced 38% fewer simulated adoptions than pushing a handful of

broadly liked items. What spreads widely is not what each user would individually choose.
- **In groups.** Aggregations that favour one enthusiastic member (most pleasure) reduce both the group's satisfaction and the spread. Average satisfaction aligns best with spread, but the most viral item differs from the group's choice in about half of the groups.

Unexpected results and cases where the approach performs poorly

- Plain MF collapsed to its biases: validation preferred heavy regularisation. MF and SocialMF are weak top-N rankers (NDCG 0.003), and popularity beats them on ranking.
- The real network spreads *less* than random graphs with the same degrees, because of clustering and reciprocity.
- Least misery did not protect the least happy member better than averaging, because the minimum of noisy predictions selects the most under-estimated items.
- The spread-aware hybrid failed to raise spread. A one-hop appeal signal does not capture multi-hop cascades.
- Unweighted neighbour pools (friends included) are worse than the bias baseline. Social signals help only when they are shrunk.
- Seeding methods become indistinguishable in the viral regime, so the advantage of CELF exists only at modest λ and small k.

11. Limitations

- **Simulator validity.** Epinions has no observed propagation events. Edge probabilities and the adoption function are modelling assumptions, λ is a scenario parameter, and the true-preference matrix is inferred from the same model families that it later judges. This circularity inflated MF's simulated satisfaction (§8.2). A held-out "truth" model of a different family, or real cascade data, would be needed for absolute claims.
- **Sample.** A 3,000-user snowball sample is denser and more hub-heavy than Epinions; spread sizes do not extrapolate. Degree-preserving rewiring lost 6% of edges as duplicates.
- **Homophily vs influence.** The observed 5.2× lift is a correlation. It may reflect shared tastes rather than influence.
- **Static setting.** There are no timestamps, so the splits are random rather than temporal. A temporal split would be a stricter leakage test.
- **Effect sizes.** RMSE differences are about 0.005–0.011. They are consistent across 3 splits but small; more splits or statistical tests per user would strengthen them.
- **Generalisation.** Group, interaction and intervention results come from one sample, one campaign item per scenario and one parameter setting each. They are illustrative comparisons, not universal rankings.

12. Reproducibility

Commands, the folder map and the requirement-to-code mapping are given in `README.md`. Run `pip install -r requirements.txt` (Python 3.11, numpy 1.26.4, scipy 1.15.3, pandas 2.2.3, matplotlib 3.10.0), then `python src/run_all.py --profile demo` (about 30 s) or `--profile full` (6.2 min). Every number in this report is in `results/full/tables/*.csv` or `results/full/summary.json`. Seeds: master 42, split seeds 0/1/2, and a fixed offset per experiment.

References. Massa & Avesani, "Trust-aware recommender systems", RecSys 2007. Kempe, Kleinberg & Tardos, "Maximizing the spread of influence through a social network", KDD 2003. Leskovec et al., "Cost-effective outbreak detection in networks" (CELF), KDD 2007. Jamali & Ester, "A matrix factorization technique with trust propagation for recommendation in social networks", RecSys 2010. Ma et al., "Recommender systems with social regularization", WSDM 2011. Koren, Bell & Volinsky, "Matrix factorization techniques for recommender systems", IEEE Computer 2009. Hu, Koren & Volinsky, "Collaborative filtering for implicit feedback datasets" (ALS), ICDM 2008. Cremonesi, Koren & Turrin, "Performance of recommender algorithms on top-N recommendation tasks", RecSys 2010. Masthoff, "Group recommender systems", Recommender Systems Handbook, 2011. Kermack & McKendrick, SIR model, 1927.